Perceptron Algorithm with AND, OR, and XOR Gates

The Perceptron is the simplest type of feedforward neural network, used for binary classification tasks.

It works well for linearly separable data but fails for non-linear cases like the XOR problem.

Algorithm Steps:

1. Initialize weights and bias to 0.

2. For each training sample, compute output:

   output = activation(weighted sum + bias)

3. Update weights and bias based on error.

4. Repeat for several iterations.

Logic Gates Tested:

- AND Gate: Linearly separable, works perfectly.

- OR Gate: Linearly separable, works perfectly.

- XOR Gate: Not linearly separable, Perceptron fails.

```
import numpy as np

# Perceptron Class Implementation
class Perceptron:
    def __init__(self, learning_rate=0.1, n_iters=10):
        self.lr = learning_rate
        self.n_iters = n_iters
        self.activation_func = self._unit_step_function
        self.weights = None
        self.bias = None

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0

        for _ in range(self.n_iters):
            for idx, x_i in enumerate(X):
```

```
                linear_output = np.dot(x_i, self.weights) + self.bias
                y_predicted = self.activation_func(linear_output)

                update = self.lr * (y[idx] - y_predicted)
                self.weights += update * x_i
                self.bias += update

    def predict(self, X):
        linear_output = np.dot(X, self.weights) + self.bias
        return self.activation_func(linear_output)

    def _unit_step_function(self, x):
        return np.where(x >= 0, 1, 0)

# Dataset for Logic Gates
X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

# Function to test Perceptron on logic gates
def test_gate(gate_name, y):
    print(f"\nTesting {gate_name} Gate:")
    perceptron = Perceptron(learning_rate=0.1, n_iters=10)
    perceptron.fit(X, y)
    predictions = perceptron.predict(X)
    print("Predictions:", predictions)
    print("Expected   :", y)
    print("Weights    :", perceptron.weights)
    print("Bias       :", perceptron.bias)

# AND Gate
y_and = np.array([0, 0, 0, 1])
test_gate("AND", y_and)

# OR Gate
y_or = np.array([0, 1, 1, 1])
test_gate("OR", y_or)

# XOR Gate (not linearly separable, Perceptron will fail)
y_xor = np.array([0, 1, 1, 0])
test_gate("XOR", y_xor)
```

Results:

- AND Gate: Predictions match perfectly.

- OR Gate: Predictions match perfectly.

- XOR Gate: Predictions fail (as expected).

Conclusion:

Perceptron is limited to linearly separable data.

For non-linear tasks like XOR, we need multi-layer perceptrons (MLPs) or advanced algorithms.